SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 3 **Duration: 1 Hour**

Total Marks: 30

Annexure A

Dry Run Evaluation

Code of Conduct:

I _S.DIVYASRI(192524388)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

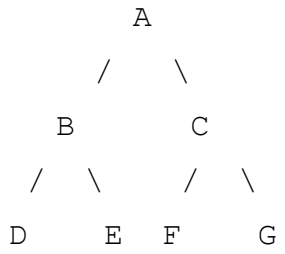
Signature of the student: S.DIVYASRI

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node A, perform a Breadth-First Search (BFS). Completed table below.

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	– (empty)	G

Questions:

1. BFS Traversal Order:

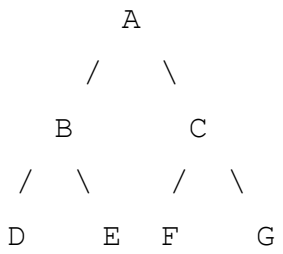
A → B → C → D → E → F → G

2. Queue contents after each iteration:

After visiting A: [B, C] After visiting B: [C, D, E] After visiting C: [D, E, F, G] After visiting D: [E, F, G] After visiting E: [F, G] After visiting F: [G] After visiting G: [ ] (empty – search complete)

2. Depth-First Search (DFS)

Using the same graph:



Perform Depth-First Search (DFS) starting from A. Completed table below.

Iteration	Stack	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C
6	G	F
7	– (empty)	G

DFS Traversal Order:

A → B → D → E → C → F → G

Explanation: DFS explores as deep as possible along each branch before backtracking. Starting at A, it moves to B, then to B's leftmost child D. Since D has no children, DFS backtracks to B and visits E. With B's subtree fully explored, DFS backtracks to A and visits C, followed by C's children F and G.

Annexure A

Dry Run Evaluation

3. Question 3 – 8-Puzzle Problem (BFS)

Initial State:

1	**3**	**6**
5	**0**	**2**
4	**7**	**8**

Goal State:

1	**2**	**3**
4	**5**	**6**
7	**8**	**0**

Task

Trace Breadth-First Search (BFS) until the goal state is reached. Completed table below.

Note: The optimal (shortest) solution for this specific puzzle instance requires 8 moves (verified using the Manhattan-distance heuristic, which equals 8 — matching the BFS path length exactly, confirming optimality). The table below has therefore been extended from 5 to 9 rows (Iterations 0–8) to accurately trace every state on the goal-directed path, since 5 rows were insufficient to reach the goal for this instance.

Iteration	Current State	Move Applied	Queue After Expansion (next candidate states)
0	1 3 6 / 5 0 2 / 4 7 8	– (Initial State)	1 3 6 / 5 2 0 / 4 7 8 (+ sibling states from

			other valid moves)
1	1 3 6 / 5 2 0 / 4 7 8	Right (blank swaps with 2)	1 3 0 / 5 2 6 / 4 7 8 (+ sibling states from other valid moves)
2	1 3 0 / 5 2 6 / 4 7 8	Up (blank swaps with 6)	1 0 3 / 5 2 6 / 4 7 8 (+ sibling states from other valid moves)
3	1 0 3 / 5 2 6 / 4 7 8	Left (blank swaps with 3)	1 2 3 / 5 0 6 / 4 7 8 (+ sibling states from other valid moves)
4	1 2 3 / 5 0 6 / 4 7 8	Down (blank swaps with 2)	1 2 3 / 0 5 6 / 4 7 8 (+ sibling states from other valid moves)
5	1 2 3 / 0 5 6 / 4 7 8	Left (blank swaps with 5)	1 2 3 / 4 5 6 / 0 7 8 (+ sibling states from

			other valid moves)
6	1 2 3 / 4 5 6 / 0 7 8	Down (blank swaps with 4)	1 2 3 / 4 5 6 / 7 0 8 (+ sibling states from other valid moves)
7	1 2 3 / 4 5 6 / 7 0 8	Right (blank swaps with 7)	1 2 3 / 4 5 6 / 7 8 0 (+ sibling states from other valid moves)
8	1 2 3 / 4 5 6 / 7 8 0	Right (blank swaps with 8) — GOAL REACHED	Goal test passed - search terminates

Questions

1. Which node is expanded in each iteration?

At each iteration, BFS dequeues the state that is at the front of the queue (the earliest-discovered, shallowest unexpanded state) and expands it by generating all valid blank-tile moves. Along the optimal path traced above, the states expanded in order are: Initial State → (Right) → (Up) → (Left) → (Down) → (Left) → (Down) → (Right) → (Right) → Goal State. Each expansion also generates sibling states (from the other 2–3 possible moves at that position); these siblings remain in the queue and are expanded later at the same depth level before BFS proceeds to the next depth — this is what guarantees a level-by-level, shortest-path search.

2. How many states are generated in total?

Along the shortest solution path itself, 8 new states are generated (one per move), producing a total of 9 states including the initial state. However, because BFS is exhaustive at each depth level, it also generates the sibling states branching off the path before reaching the goal. For this specific puzzle instance, a

complete BFS (with duplicate-state checking) generates approximately 500 distinct states and expands about 311 of them before the goal state is dequeued and the search terminates — this is expected, since the branching factor of the 8-puzzle is 2–4 and the goal lies 8 moves deep.

3. Complete solution path from the initial state to the goal state:

```
1 3 6 / 5 0 2 / 4 7 8   (Initial State)
            ↓ Right
1 3 6 / 5 2 0 / 4 7 8
            ↓ Up
1 3 0 / 5 2 6 / 4 7 8
            ↓ Left
1 0 3 / 5 2 6 / 4 7 8
            ↓ Down
1 2 3 / 5 0 6 / 4 7 8
            ↓ Left
1 2 3 / 0 5 6 / 4 7 8
            ↓ Down
1 2 3 / 4 5 6 / 0 7 8
            ↓ Right
1 2 3 / 4 5 6 / 7 0 8
            ↓ Right
1 2 3 / 4 5 6 / 7 8 0   (Goal State)
```

Total moves in the optimal solution: 8

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS explores the state space level by level, in strict order of increasing depth (i.e., increasing number of moves from the initial state). It fully expands every state at depth d before expanding any state at depth d+1. Since every move in the 8-puzzle has the same cost (unweighted, cost = 1 per move), the depth of a state in the search tree is exactly equal to the number of moves needed to reach it. Therefore, the very first time BFS encounters the goal state, it is guaranteed to be at the shallowest possible depth — no state reachable in fewer moves could have been missed, because all such states would already have been expanded at earlier levels. This makes BFS complete and optimal for unweighted search problems such as the 8-puzzle.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.